FUNCTIONS

CS1910

Computer Science I

EXAMPLE CODE

- https://colab.research.google.com/github/UPEI-CS/CS1910-Students/blob/main/lectures/08-Functions/CS1910_Functions_Workbook.ipynb

FUNCTIONS

- We have been using functions since the start of the course
- Examples: input(…), print(…), len(…), abs(…)
- We can write our own functions
- This is another form of abstraction
- Recall from Computational Thinking
  - Abstraction is a mechanism to filter out specifics and generalize a process
  - The print method allows us to display information on a console but we don't really know how it works
    - It accepts various common data types, lengths of output, etc

FUNCTIONS

- Group a set of command into a single reusable block of code
- The code can be parameterized
  - i.e., it can accept details provided as variables
- Simplifies code
- Aids understanding, readability and maintenance
- Increases efficiency
- Hides complexity

FUNCTIONS

- Can “return” or produce values
- Variables declared within a function belong to the function (their scope is restricted to within the function
  - Variable names within a function do not obscure or collide with those from other areas of your code
- When we call a function we may pass an argument to the function, but in the function definition we call those parameters

COHESION

- The degree to which the code within a function (or grouped in general) fits together and contributes to a single purpose (singular responsibility)
- High cohesion: code within a function accomplishes one task
  - Leads to easier testing, higher maintainability and higher readability and reuse
- Low cohesion: code within a function accomplishes many tasks or unrelated tasks
  - Leads to more brittle solution, less reuse and less ability to maintain and understand

Our goal →

Determining where to introduce functions and how to define them to improve cohesion is a skill that can be improved with practice

EXAMPLE: DRAW A POLYGON

- Recall from our prior class examples drawing a regular polygon
- We used aspects of computational thinking to create a beautiful program:

If I want to draw different shapes I need to change these variables

If I want to draw 2 shapes I need to cut and paste this code multiple times

```
# some variables to make our solution more robust (abstraction)
FULL_CIRCLE = 360 #ALL_CAPS denotes constants (things that never change)
length = 100
sides = 6
angle = FULL_CIRCLE / sides

#patterns to break down the shape into smaller bits
for side in range(sides):
    forward (length)
    right(angle)
```

DRAW A POLYGON

- Reuse the code by wrapping it into a function
- Parameterize the solution so we can draw different shapes by specifying what we want when we do the drawing

Parameters that that function accepts

Pseudo Code:

```
FUNCTION polygon (sides, length)
    angle = 360 / sides
    for side in 0 to sides:
        forward(length)
        right(angle)
END FUNCTION
```

Defining the function in pseudo-code

Arguments passed to the function (take the place of the respective parameters)

```
polygon(6, 20) #draw a 6-sided polygon with side length 20
polygon(10, 6) #draw a 10-sided polygon with side length 6
```

Calling the function

DRAW A POLYGON

Pseudo Code:

```
FUNCTION polygon (sides, length)
    angle = 360 / sides
    for side in 0 to sides:
        forward(length)
        right(angle)
END FUNCTION

polygon(6, 20)
polygon(10, 6)
```

similar code without functions

```
sides = 6
length = 20
angle = 360 / sides
for side in 0 to sides:
    forward(length)
    right(angle)

sides = 10
length = 6
angle = 360 / sides
for side in 0 to sides:
    forward(length)
    right(angle)
```

Repeated code is something to watch for (Patterns) to indicate that functions can offer an improvement

Repeated code is double the maintenance

EXAMPLE: CALCULATE THE AREA OF A RECTANGLE

- Math formula: area = length x width

Pseudo Code:

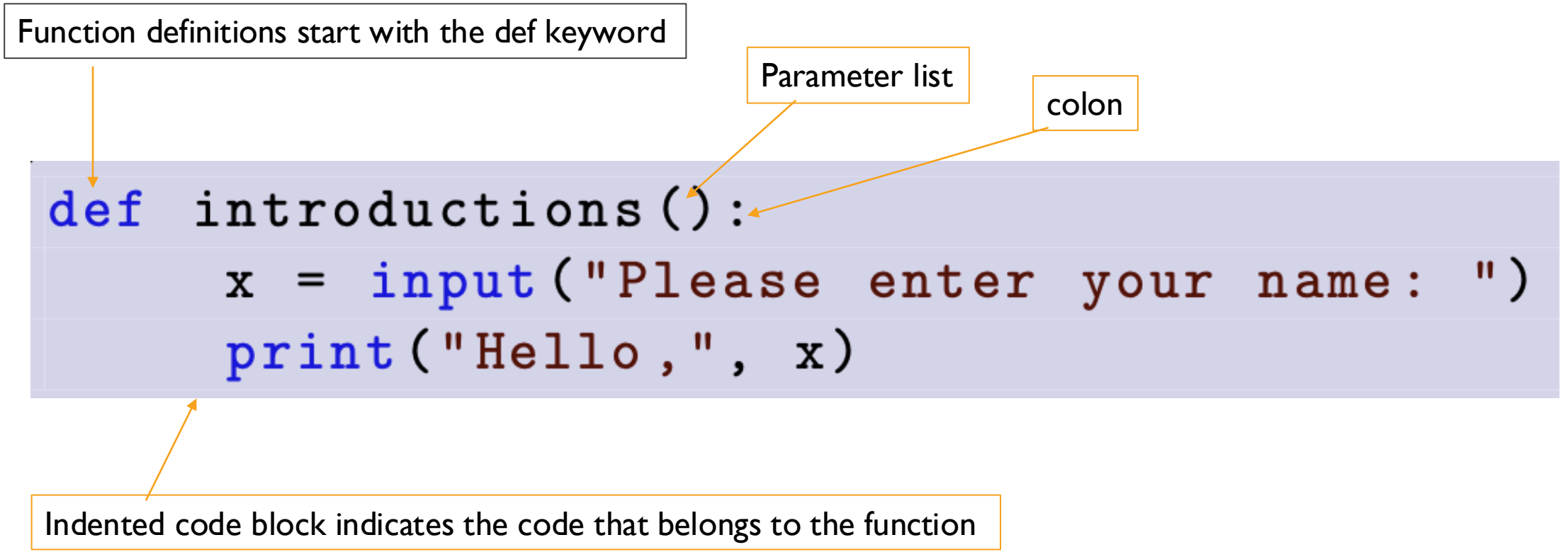
```
FUNCTION rectangle_area (length, width)
    area = length * width
    RETURN area
END FUNCTION

length = 10
area1 = rectangle_area(length,20)
area2 = rectangle_area(5,6)
```

Send this value back to the calling site

This variable is unrelated to the length parameter

Calling site



PYTHON FUNCTION SYNTAX

FUNCTION WITH PARAMETERS

```
# defines the function only:
def introductions(greeting):
    print(greeting)
    x = input("Please enter your name: ")
    print("Hello,", x)

# this function call actually calls the function,
# which executes its code.
introductions("Welcome to my Python program!")
```

```
Welcome to my Python program!
Please enter your name: Chris
Hello, Chris
```

FUNCTION WITH RETURN VALUE

```
# defines the function only:
def introductions(greeting):
    print(greeting)
    x = input("Please enter your name: ")
    print("Hello,", x)
    return x

# this function call actually calls the function,
# which executes its code.
username = introductions("Welcome to my Python program!")
```

*Functions that do not have a return statement will return the value None

DEFINE THEN CALL

- A function must be defined before it is called

```
# this fails -- function called before definition
username = introductions('Welcome to my Python program!')

def introductions(greeting):
    print(greeting)
    x = input("Please enter your name: ")
    print("Hello,", x)
    return x

# this is fine - function called after definition.
username = introductions("Welcome to my Python program!")
```

*Note that colab can produce some anomalies if you rerun code cells
Or edit code cells that are above other cells after those lower cells have been run

SELF-CHECK

```
def fireball_damage():
    damage = 30
    return damage

damage = 0
D = fireball_damage()
print(damage)
```

What is the output?

SELF-CHECK

```
def fireball_damage():
    damage = 30
    return damage

damage = 0
D = fireball_damage()
print(damage)
```

This variable lives inside the function only (its scope is just within the function block)

What is the output?

0

```
def print_name(name):
    print(f"my name is: {name}")

print(print_name("Bob"))
```

CLASS PARTICIPATION

What is wrong with the code?

GENERALIZING CODE

- Consider the following code to find spaces in a string

```
# counting spaces in a specific string

my_string = "If you wish me luck, don't tell me to break a leg"

space_count = 0
for char in my_string:
    if char == " ":
        space_count += 1

print(f"spaces: {space_count}")
```

GENERALIZING

Count spaces in any string

```
# counting spaces in any string as a method

def count_spaces(s):
    space_count = 0
    for char in s:
        if char == " ":
            space_count += 1
    return space_count

my_string = "If you wish me luck, don't tell me to break a leg"
space_count = count_spaces(my_string)
print(f"spaces: {space_count}")

other_string = "I will return to the city of lakes"
space_count = count_spaces(other_string)
print(f"spaces: {space_count}")
```

GENERALIZING

- Generalization is the process of modifying a function / algorithm so that it can solve a wider range of problems
  - Can we further generalize our counting spaces solution?

GENERALIZING

Count any character in any string

```
# generalizing our solution to count any specified
# character in any string
def count_chars(string, ch):
    char_count = 0
    for char in string:
        if char == ch:
            char_count += 1
    return char_count
```

FIND A PARTNER

Consider the following code

What patterns do you see?

Can you use functions to improve it?

```
# get hours and minutes from the user

hour = 0
while not (1 <= hour <= 12):
    hour = int(input("Enter the hour (1-12): "))

min = 0
while not (1 <= min <= 59):
    min = int(input("Enter the min (1-59): "))

print(f"Time: {hour} o'clock and {min} minutes")
```

MORE FIND A PARTNER

- Find all prime numbers between 25 and 75
- The following code can print out if 8 is prime or not:

```
if 8 % 2 == 0:
    print("not prime")
elif 8 % 3 == 0:
    print("not prime")
elif 8 % 4 == 0:
    print("not prime")
else:
    print("prime")
```

Improve this approach through functions to generalize a solution

DOCUMENTING FUNCTIONS

- Triple quotes denote a docstring “””” or ””
- Docstrings are used for general purpose documentation to other programmers who may want to make use of your code (as opposed to comments that might explain inner workings of code)
- A docstring should be used on all methods to outline what the method does and how to use it
  - Each parameter should be documented
  - The return value and type should be documented

DOCUMENTING FUNCTIONS

- Imagine if we had to read the implementation of function to understand how to use them …
- Other programmers should be able to read a few lines of documentation to understand what your function does and what type of arguments you can pass to it

```
def combined_size(a, b):
    return len(a) + len(b)
```

Without reading the code someone might expect this to work with integers or floating point values

Reading the code to determine how to use something, is not a reasonable ask (nor desired) Akin to disassembling your microwave and examining the circuitry to know how to heat up your food

DOCUMENTATION

```
# documenting the count_chars method
# 1. document what the method does
# 2. document each of the parameters
# 3. document the return value

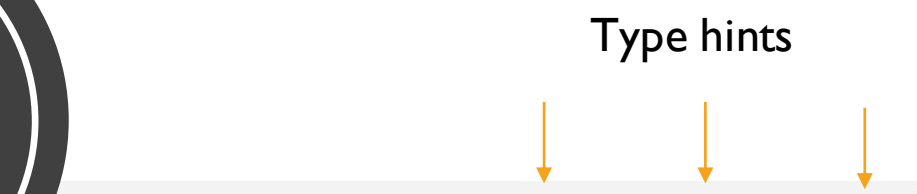
def count_chars(string, ch):
    '''
        count and return the number of occurrences of ch in string
        Parameters:
            string: any string
            ch: a string with length 1

        return: the count of number of occurrences of single character ch within string
    '''
    char_count = 0
    for char in string:
        if char == ch:
            char_count += 1
    return char_count
```

TYPE HINTS

- We can add a form of documentation called a hint to our methods (to any variable actually)
- Indicate what type of variable is expected when using a function (or what the return value is) int, float, str, etc
- Use the semi-colon after the variable name to indicate a type hint
- For this course we mostly focus on built-in types like int, bool, float, str, etc
  - More advanced forms of type hinting are available

EX: TYPE HINTS




```python
def count_chars(string : str, ch: str) -> int:
    '''
        count and return the number of occurrences of ch in string
        Parameters:
            string: any string
            ch: a string with length 1

        return: the count of number of occurrences of single character ch within string
    '''
    char_count = 0
    for char in string:
        if char == ch:
            char_count += 1
    return char_count
```

DEFAULT ARGUMENTS

- Default arguments allow users to use your function without supplying some of the arguments.
- In the case that an argument is omitted its default value is used
- Default arguments must be placed at the end of the argument list
  - Once a default argument is intended all the rest of the arguments are left off as well
- Improve function flexibility by allowing them to be called with fewer arguments

DEFAULT ARGUMENTS

If ch is not supplied as an argument “ ” is used

```
def count_chars(string, ch= " "):
    '''
        count and return the number of occurrences of ch in string
        Parameters:
            string: any string
            ch (optional): a string with length 1 (default: " " (space))

        return: the count of number of occurrences of single character ch within string
    '''
    char_count = 0
    for char in string:
        if char == ch:
            char_count += 1
    return char_count


count_chars("hello world!")
```

KEYWORD ARGUMENTS

- So far we have used what is called position arguments in our function calls
- We supply the arguments left to right that match the parameters left to right (implicitly)
- We can also supply arguments using the explicit parameter name of the argument

```python
def greet(name, message="Hello"):
    print(f"{message}, {name}!")

# Using positional arguments
greet("Alice")  # Output: Hello, Alice!

# Using keyword arguments
greet(name="Bob", message="Hi")  # Output: Hi, Bob!
```

KEYWORD ARGUMENTS

- Can improve readability
  - For long argument lists explicitly stating the parameter names can aid readability
- Can improve flexibility with default arguments
  - By explicitly naming some arguments you can leave others to default values with increased flexibility
- Can be combined with positional arguments
  - BUT positional arguments must come first

```
# Mixing positional and keyword arguments (positional first)
greet("Charlie", message="Greetings") # Output: Greetings, Charlie!

# Invalid: Keyword argument before positional
# greet(message="Hey", "David") # This would raise a SyntaxError
```

```python
def print_details(*, name, age):
    print(f"Name: {name}, Age: {age}")

# Valid: Must use keyword arguments
print_details(name="Eve", age=30)

# Invalid: Positional arguments are not allowed for keyword-only
# print_details("Frank", 40) # This would raise a TypeError
```

KEYWORD ARGUMENTS

Can force the use of keyword arguments by placing an asterisk in the function definition

TRY AT HOME

- Convert our old turtle draw program to draw a square into a method (how can it be generalized?)
- Write a method that returns a reversed version of a string, i.e., if the input is "Madam" it will return "madaM"
- Write a method that converts an integer between 0 and 10 to the appropriate word for that value, e.g., 1 converts to "one”
- Write a program to repeatedly ask the user to input single letters (“A”, “B”, “C”,etc) but stop the program when the user enters a letter that they entered previously. Use methods with high cohesion.
  - No list or dictionary required: concatenate the letters in a string

MORE TRY AT HOME

- Write a method to ask the user to enter ‘Y’ or ‘N’, return their choice
- Find an old example from class, determine where and/or if methods could be used to improve the example
- Create a method to determine the middle number from 3 numbers passed as arguments to the method, it should return the middle number. Use keyword arguments when calling the method